Otros mercados y otros timeframes

AlgoProject — manual de uso. Actualizado el 2026-10-02 con `tools/manual.py`.

1. 30. Retest en otros mercados — ¿el edge sobrevive fuera de su activo?
2. 5. Retest en mercados adicionales — ¿el sistema gana por acertar CUÁNDO entra, o por estar comprado?
3. 39. Crossmarket por lotes — el paso 10 sin abrir el panel
4. 31. crossTF — ¿el edge aguanta si lo miras en un reloj más lento?

30. Retest en otros mercados — ¿el edge sobrevive fuera de su activo?

Qué pregunta responde

Una estrategia que gana dinero en el oro puede estar capturando algo real del mercado, o puede haberse ajustado a las casualidades de esos años concretos de oro. La forma más directa de distinguirlo es llevarla a otro mercado al que nunca se ajustó y ver si sigue viva.

Esta página cubre la parte mecánica: **qué mercados, desde cuándo y con qué costes** se configuran en el crosscheck *Retest on additional markets* de SQX.

Cuándo lo usas, y cuándo no

Después de que la población haya pasado la puerta OOS (`05-cribado-oos.pdf › 29-puerta`). Antes no: este test es caro y la puerta es lo que lo hace asequible.

No lo uses para *buscar* en qué mercados funciona una estrategia. Los mercados están fijados de antemano en `assets/_markets.yaml`, y elegirlos después de ver los resultados convierte la prueba en una selección.

Qué mercados hay declarados

```
python3 -m sqx.projects.crossmarket XAUUSD
```

```
XAGUSD_DukasM1_Infinox   family   2008.01.01 a 2022.12.31   costes: ⚠ NINGÚN fichero de
assets/symbols/ declara este feed
BRENTCMDUSD_ftmo         family   2008.01.01 a 2022.12.31   costes: ⚠ ...  ⚠ data_from sin
averiguar
```

Dos categorías, y la diferencia importa:

- **`family`** — mismo motor económico que el activo principal. Es la prueba fácil: pasarla no demuestra mucho, fallarla sí dice algo.
- **`structural`** — misma *forma* de mercado (volatilidad, horario, ruido) sin motor compartido. Es la prueba dura.

La ventana de cada mercado

La declara `assets/_build.yaml` de una vez para todos los activos, en `crossmarket.segment`: **`build..oos1`** (decisión del dueño, 2026-09-24), con la misma notación de dos puntos que la MC Retest. O sea, desde el **inicio del IS del activo principal** —o desde la primera barra de ese mercado, la que sea más tardía— hasta el final de `oos1`. Para el XAUUSD: la plata va de 2008 a 2022, y un mercado cuyos datos empiezan en 2013 va de 2013 a 2022.

Se usa todo el histórico a propósito. Recortarlo a la ventana del activo principal tira justo los años que responderían a la pregunta.

Los costes

Cada mercado extra lleva su propio `<Setup>` dentro del crosscheck, con **su** spread, **su** slippage, **su** comisión y **su** swap, leídos de **su** `assets/symbols/<SIM>.yaml`.

⚠ **El comando se niega a escribir nada si un mercado declarado no tiene fichero de costes.** No es un fallo: SQX lo correría tan tranquilo con el spread por defecto de su registro, y un resultado cross-market con un coste inventado es peor que no tener resultado.

Un `<Setup>` lleva un solo spread y la ventana cruza los dos tramos, así que se cobran los del **OOS**. Al mercado que tiene que sorprendernos no se le da nunca el precio barato.

`<MainTestValues>` dice qué se hereda del test principal en vez de tomarse de este Setup: el timeframe, la precisión y la distancia mínima se comparten; las fechas, el spread, el slippage, las comisiones y el swap son propios. La sesión se hereda, porque `assets/` no declara sesión para un mercado de crosscheck.

Cómo se configura

```
python3 -m sqx.projects.crossmarket XAUUSD \
    --cfx ~/Desktop/SQX_w2/user/projects/<P>/project.cfx \
    --task Retest-Task3.xml --timeframe M30
```

`Retest-Task3.xml` es la tarea de mercados adicionales del donante; lee y escribe el databank `Retest Markets - Family`. Ejecutarla es igual que cualquier otra tarea del custodio: ver `04-sqx-plantillas-y-proyectos.pdf › 28-builder` y la skill `/template-run`.

Aquí no se filtra nada

`crossmarket.conditions: []` en `assets/_build.yaml`, decisión del dueño del 2026-09-24. El comando lo hace cumplir: apaga **todas** las condiciones de aceptación del crosscheck, fuerza `DeleteFailedStrategies` a false y dice cuántas apagó.

```
  XAGUSD_DukasM1_Infinox         family      2008.01.01 a 2022.12.31
  BRENTCMDUSD_ftmo               family      2013.01.01 a 2022.12.31
1 condiciones de aceptacion apagadas — esto es evidencia, no un filtro
```

El donante traía una viva, `ReturnDDRatio > 1` sobre el primer mercado. Con una condición viva SQX **no escribe en el databank de salida** la estrategia que no la cumple, y entonces el análisis de Python se queda sin las muertas — que son la mitad de la evidencia: sin ellas no se puede decir qué mercado mató a qué estrategia. SQX corre los backtests; el veredicto se toma en Python y se aplica con `/curate`.

 Con cero condiciones vivas, `<MinConditions>` y `<MinMarkets>` quedan inertes y se dejan como estaban. En la primera corrida real, comprueba que el databank de salida tiene tantas estrategias como el de entrada y apunta lo que veas en `knowhow/conditions/crossmarket-crosstf-no-conditions.md`.

Cómo se lee el resultado

Y di a qué categoría pertenecía cada mercado. Pasar el test de `family` puede significar sólo que los dos mercados son el mismo trade.

Qué NO demuestra

Que una estrategia pase aquí no la valida: la valida el protocolo entero. Que falle no la rompe necesariamente —puede que ese mercado tenga una microestructura distinta— pero sí obliga a explicar por qué se conserva.

5. Retest en mercados adicionales — ¿el sistema gana por acertar CUÁNDO entra, o por estar comprado?

Qué pregunta responde

Una estrategia long-only que opera en un mercado que sube gana dinero casi haga lo que haga. Eso no es habilidad: es estar comprado. La pregunta que responde este módulo es otra, y es la única que importa:

> Si cogemos exactamente el mismo ritmo de operar —el mismo número de operaciones, la misma duración de cada una, los mismos huecos entre ellas, el mismo día de la semana y la misma hora— y lo colocamos en momentos **al azar** del mismo mercado, ¿habría ganado lo mismo?

Se generan miles de esas versiones al azar y se mira dónde queda la de verdad. Si la de verdad queda entre las mejores, sus entradas llevan información. Si queda en el montón, lo que había era la deriva del mercado y nada más.

Se hace sobre mercados que la estrategia **nunca vio** cuando se optimizó. Ahí sus entradas son efectivamente ciegas, y batir al azar en varios de ellos es algo que el sobreajuste no puede fabricar.

Cuándo lo usas, y cuándo no

Lo usas después de haber pasado tus estrategias por un retest multi-mercado en SQX, para decidir cuáles siguen adelante.

No lo usas para saber si una estrategia está sobreajustada al oro. No lo detecta, y decirlo al revés es el error más fácil de cometer con este informe. Lo que mide es si el acierto **se traslada** a otros mercados.

No lo usas sobre el propio activo base. La estrategia se optimizó ahí, así que gana a lo aleatorio por construcción: en la prueba de este módulo sobre oro, las ocho estrategias probadas salieron con p entre 0.0002 y 0.007. Eso no dice nada bueno de ellas, sólo dice que el código funciona. Por eso el activo base no es un mercado más: sale en el informe como **referencia**, nunca como evidencia.

No esperes un veredicto. Este módulo no dice MANTENER ni DESCARTAR, y no esconde ningún mercado. Te da los números y, al lado de cada uno, la lista de motivos para desconfiar de él. La decisión la tomas tú. Antes no era así: un mercado con menos de 30 operaciones o con órdenes pendientes se caía entero del análisis, y eso costó tirar un resultado de Brent con p = 0,005 por culpa de un 8% de entradas que no caían en apertura de vela.

Antes de empezar

Tres cosas, en este orden:

1. **En SQX**, pasa tus estrategias por una tarea de retest sobre mercados adicionales, y deja el resultado en una databank. Los mercados que elijas ahí son los que manda: el módulo **descubre** de la exportación en qué mercados se retesteó de verdad, leyendo la columna `Symbol` de las propias operaciones. `assets/_markets.yaml` sólo les pone categoría y nombre.

2. **Exporta en cuanto termine.** Varias databanks se vacían en cada ciclo de la cadena de tareas, y la sincronización horaria borra del disco lo que no está en memoria. Si el retest queda ahí una noche, puede no estar por la mañana.
3. **Fija la lista de mercados antes de mirar resultados.** Elegir los mercados después de ver dónde funciona convierte la prueba en una selección y los p-valores en decoración.

Comprueba lo que vas a aplicar de costes:

```
python3 -m core.assets XAUUSD
```

Que salga `UNDECIDED` **no bloquea este módulo**, y es la única excepción a esa regla en todo el proyecto. El motivo: aquí no se autoriza nada, se reproduce lo que SQX ya simuló. El coste se recupera operación a operación de los propios datos exportados (`precio bruto − beneficio reportado`), no se elige. En el oro eso mide 8 $ por lote y lado, que es exactamente lo que SQX tiene configurado.

Dónde se clasifican los mercados

En `assets/_markets.yaml` , un bloque por activo base y, dentro, una lista por categoría:

```
XAUUSD:
  main: XAUUSD_DukasM1_Infinox
  timeframe: M30
  out_of_sample: {from: 2018-01-01, to: 2022-12-31}
  categories:
    family:
      - {feed: XAGUSD_DukasM1_Infinox, data_from: 2003-08-08}
      - {feed: BRENTCMDUSD_ftmo, data_from: unknown}
    structure: []
```

Este fichero clasifica; no decide qué existe. Lo que se analiza sale de la exportación: la columna `Symbol` del `trades.parquet` que dejó el paso 2 dice qué mercados hubo, así que no puede mentir. (Hasta el 23-09-2026 el paso 2 dejaba una carpeta `trades/<mercado>/` con un CSV por estrategia; ahora es un solo parquet con todos los mercados dentro, 118 ficheros → 1.) Si un mercado aparece en el export y no está aquí, se analiza igual y sale marcado como `sin clasificar` . Si está aquí y el export no trae operaciones suyas, el comando lo imprime al arrancar como ausente. Antes, un desajuste entre los dos salía como un mercado con cero estrategias, que no parece un error y lo es.

`feed` tiene que estar escrito **exactamente** como lo llama SQX. `data_from` es la primera fecha con datos de ese mercado: no filtra nada, está para que veas cuánta ventana común te queda de verdad. Las categorías (`family` , `structure` , y las que vengan) son etiquetas: salen en las tablas y no cambian ningún cálculo.

La lista sigue fijándose **antes** de mirar resultados. Elegir mercados después de ver dónde funciona convierte la prueba en una selección.

`out_of_sample` es otra cosa: es el tramo OOS del backtest **principal**, copiado del `<OutOfSample><Range/>` del propio proyecto (en el XAUUSD está dentro de `Build-Task3.xml` , y vale `2018.01.01 → 2022.12.31`). Sobre él se corre el mismo test de entrada aleatoria, en la pestaña **Entrada aleatoria · OOS principal**.

Hay que escribirlo a mano, porque el export no lo sabe. Todas las operaciones que exporta el retest salen marcadas `Sample type = IST`, estén dentro o fuera del tramo — comprobado en los tres mercados de `Retest_Markets_-_Family` —, así que el corte no se puede leer de los datos y las fechas son el único testigo que hay. Si un activo base no declara `out_of_sample`, la pestaña sale vacía diciéndolo; no se inventa nada.

Cómo se ejecuta

Dos comandos preparan los datos y el tercero corre la prueba. El tercero tiene dos formas: el databank entero con un veredicto por estrategia, o **una estrategia en profundidad** con todas las pestañas. La ventana de escritorio (`ui/`) llama a esta misma segunda forma y pinta su resultado; el panel del navegador (`explorer.serve`) se retiró el 25-09.

```
# 1. Las barras de todos los mercados: la librería M1, y el resto de timeframes se calcula al
vuelo
python3 -m sqx.export.sync_bars

# 2. Los trades del retest, partidos por mercado (~4 min por cada 200 estrategias)
python3 -m sqx.export.export_retest --project XAUUSD --databank "Retest Markets - Family"

# 3a. El databank entero: amplitud entre mercados y un veredicto por estrategia
python3 -m studies.transfer.crossmarket.report --project XAUUSD \
    --databank "Retest Markets - Family" --asset XAUUSD --day 2026-09-14

# 3b. Una estrategia, con todas las pestañas (y, con --only, un solo mercado)
python3 -m studies.transfer.crossmarket.report --project XAUUSD \
    --databank "Retest Markets - Family" --asset XAUUSD --day 2026-09-14 \
    --strategy "Strategy 24.14.35"
```

flag	obligatorio	qué hace
`--asset`	sí	activo base. Es la clave que se busca en `assets/_markets.yaml`
`--project`	sí	proyecto en el master
`--databank`	sí	la databank donde dejaste el retest
`--day`	no (paso 3)	la fecha del export del paso 2, `AAAA-MM-DD`; hoy si no se pone
`--limit`	no (paso 2)	exporta una muestra aleatoria reproducible de N estrategias en vez de todas
`--strategy`	no (paso 3)	estudia esa estrategia entera en vez del databank
`--only`	no (paso 3, con `--strategy`)	sólo ese mercado, por su nombre de feed
`--floor`	no (paso 3a)	fracción de mercados que debe tener esperanza positiva para MANTENER. Lo mismo que `--set verdict.breadth_floor=`; por defecto 0,5
`--set`	no (paso 3)	cambia un knob de `config.yaml` para esta ejecución, p. ej. `--set nulls.draws=20000`
`--workers`	no (paso 3)	procesos en paralelo. Por defecto, todos los núcleos

Los pasos 1 y 2 **arrancan SQX** (el worker, nunca el master) y se pueden ejecutar con tu interfaz abierta. El paso 3 no toca SQX: sólo lee ficheros.

Qué produce

En `~/Desktop/AlgoData/reports/<proyecto>/<databank>/<export>/crossmarket/`:

archivo	qué es
`crossmarket.html` / `.md` / `.json`	3a: la pestaña **Amplitud** — cuántos mercados sostienen cada estrategia
`verdict.csv`	3a: `strategy`, `identity`, `verdict` (MANTENER / DESCARTAR) y el motivo, para `/curate`
`estrategias/<nombre>.html` / `.json`	3b: el estudio completo de una estrategia, con las pestañas de abajo
`manifest.json`	qué export se leyó, con qué configuración entera

El precio es real — unos **107 segundos por estrategia** con las 25.000 tiradas por defecto sobre dos mercados en 3b. En 3a el comando imprime `PROGRESS <n>` y una línea por estrategia:

```
PROGRESS 60 Strategy 47.20.34 MANTENER — 1 de 2 mercados con la esperanza por encima de cero (50% >= 50%)
PROGRESS 63 Strategy 30.9.23 DESCARTAR — solo 0 de 2 mercados con la esperanza por encima de cero (0% < 50%)
...
5 de 30 pasan el suelo de amplitud -> .../crossmarket/verdict.csv
```

Un mercado en el que esa estrategia **nunca disparó** sale como `sin operaciones`. El export sólo escribe el fichero de un mercado si hubo operaciones ahí, así que esa ausencia es un resultado sobre la estrategia, no un dato que falte.

El cajón de configuración

Los 46 knobs de `config.yaml`, agrupados por sección; cada uno tiene su frase en `tooltips.py`, que es lo que enseña el cajón de la ventana al pasar el ratón. Se cambian con `--set` o desde ese cajón, y afectan sólo a esa ejecución: el `config.yaml` no se reescribe.

grupo	lo que controla
Modelos nulos	cuántos backtests aleatorios, la semilla, el bloque de régimen, qué modelos correr, si se replica el cierre del viernes, y el tamaño de lote
Barrido de ventana	qué tamaños de bloque, qué modelos se barren, cuál es la referencia, y los umbrales de potencia: meses mínimos, operaciones mínimas por bloque, hueco libre mínimo y qué parte de las operaciones puede caer en bloques débiles
Estratos de régimen	sólo para Regime Strata: cuántos cuantiles de ATR y cuántas velas para el signo de la tendencia
Cuenta y curvas	la cuenta de partida, cuántos puntos tiene cada curva, qué percentiles dibuja el cono, y **la caída objetivo a la que se reescala cada mercado** para compararlos a riesgo igual
Bootstrap	tiradas, operaciones por bloque y percentiles de los intervalos de confianza
Exposición (1c)	el t mínimo de la deriva para marcar E como no interpretable, las velas por bloque del intervalo de Fieller, y qué hacer con las operaciones de MFE cero
Test pareado (1b)	el lado del test de Wilcoxon, **cómo se define «el mismo tramo de mercado»** (ventana centrada de ±N meses, o la partición en semestres) y con qué otras definiciones se corre además
Portfolio	semanas de calendario por bloque del remuestreo, cuántas tiradas, y operaciones por bloque al barajar el orden
Coste y ejecución	múltiplos de coste, desplazamiento en velas, fracciones de slippage, los parámetros de la ejecución degradada, y **si se calibran desde `execution.yaml`**
Lectura y avisos	alpha, los umbrales que disparan un aviso y la correlación supuesta

Ninguno de esos umbrales decide nada. `alpha`, `min_trades`, `min_on_open`, `fill_tolerance` y `max_fill_error` sólo colorean números y disparan avisos; ningún mercado se cae por ellos.

Las doce pestañas

pestaña	qué muestra
Backtest	**La que abre.** Lo que hizo de verdad cada backtest: beneficio, caída en dólares y en %, Ret/DD, Sharpe, PF y racha perdedora, con el activo base aparte y marcado como referencia. Debajo, **la misma comparación a riesgo igualado**; el **p conjunto** de todos los mercados fuera de muestra; **por dónde salieron las operaciones** y cuánto del beneficio descansa en cada salida; las **curvas de capital de todos los mercados solapadas**, cada uno con su cuenta independiente y en sus fechas reales; la matriz de correlación; lo que dijo cada test; lo que sostiene esos números (MinTRL y los CI); los avisos; las comprobaciones mecánicas; y un párrafo explicando cada estadístico
Entrada aleatoria (1a)	Sub-pestañas por **mercado**, y debajo por **modelo**. Dentro: la **curva de equity real sobre el cono de las aleatorias** y, a su lado, el histograma del indicador que elijas, con la mediana y el CI 95% marcados, su tabla de valores y el % de simulaciones que el real bate. Debajo, la tabla completa de percentiles
Entrada aleatoria · OOS principal	**El mismo test 1a, sobre el tramo OOS del backtest principal y nada más.** Un cono de equity y una tabla completa por cada modelo nulo, y debajo los avisos de esa fila. Es el único sitio donde el activo base cuenta como algo más que referencia — y el primer párrafo de la pestaña explica por qué tampoco ahí es dato virgen
Modelos	El mismo p bajo las cuatro formas de aleatorizar (cinco con Regime Strata), y qué cambia cada una
Barrido de ventana	Arriba, una **rejilla de mercados × tamaños de bloque** con el p de cada celda y la tendencia al lado. Pulsas un mercado y debajo se abre el suyo — los tres modelos de colocación libre en **una sola curva**, con Calendar Shift como línea fija, la tabla de potencia, un histograma por tamaño de bloque y el cono de equity del tamaño que elijas
Pareado (1b)	La pregunta explicada arriba del todo, el alfa de timing **en cinco unidades** (bps, %, R, $ por operación y **$ acumulado**), el p de Wilcoxon, y debajo **la tabla de sensibilidad**: el mismo test con ventanas centradas de ±3, ±6 y ±12 meses y con la partición en semestres
Exposición (1c)	La pregunta explicada y **en qué se diferencia de 1b**. A por unidad de riesgo preside; E sale siempre con su **intervalo de Fieller**, que dice «no acotado» cuando el mercado no tiene deriva, y con el % de réplicas en las que el denominador cambiaba de signo
Coste y ejecución	El múltiplo de coste de equilibrio, **de dónde sale cada supuesto** (y el coste que SQX cobró de verdad frente al que dice `execution.yaml`), y las mismas operaciones ejecutadas peor 25.000 veces: cono de equity e histogramas
Huella	Qué mide cada métrica, explicado una por una, y **cuatro histogramas solapados por mercado** contra el activo base: duraciones, retornos por operación, MAE/ATR y MFE/ATR. Más la captura de MFE, que mide la salida
Portfolio	Todos los mercados en **una sola cuenta**, el oro incluido. Qué aporta o qué resta cada mercado (Δ sobre la cartera entera), cuánto tiempo hubo dos o más posiciones abiertas, y dos formas distintas de preguntar cuánta suerte hay: remuestreo por bloques de calendario y barajado del orden
Avisos	Cada motivo de desconfianza, en **cuatro partes**: qué es y qué número lo disparó, **a qué afecta**, **a qué NO afecta**, y qué hacer. **Nada se excluye por esto**
Glosario	Qué significa cada número

(El barrido de ventana sí tiene capturas, más abajo, hechas con el panel ya retirado; el contenido es el mismo. Las de las pestañas Backtest, Huella, Portfolio y Entrada aleatoria · OOS principal están pendientes: la regla 8 exige que sean de una ejecución real, así que las pega quien abra la estrategia en la ventana la próxima vez.)

Qué desapareció, y por qué

- **Resumen**, **Significancia** y **Correlación** eran tres pestañas para una sola pregunta. Ahora son la pestaña **Backtest**, que es la que abre.
- **El mercado** (Hurst, variance ratio, ADX, eficiencia) se ha quitado. Esos números existían para una regresión que necesita **seis mercados o más**; con dos, o con cuatro, no puede ajustarse.
- **El PCA** de la correlación se ha quitado: con dos o tres curvas, PC1 es casi una función de la correlación media, así que no añadía ningún eje que la matriz no enseñara ya.

La tabla de riesgo igualado

Un mercado que gana el doble sufriendo el triple **no lo hizo mejor**: arriesgó más. Esa tabla multiplica el tamaño de posición de cada mercado hasta que su peor caída es exactamente el 10% de la cuenta (`equity.risk_target_dd`), y multiplica su retorno por ese mismo factor.

Su punto débil está escrito debajo de ella en la propia pestaña: el peor drawdown es **un** momento de la muestra, así que el número es ruidoso. Se lee junto al **Ret/DD**, que usa los mismos dos números sin depender del tamaño de la cuenta.

La pestaña Portfolio

Contesta lo que el retest deja abierto: *vale, el oil no es una maravilla, pero ¿me rompe la cartera?*

Todos los mercados se juntan en **una sola cuenta de 100.000 $**, con el oro dentro como posición núcleo, y la caída se calcula sobre la curva **combinada** — nunca sumando las de cada mercado.

El gráfico de equity de esta pestaña no es el de la principal. Aquí la línea gruesa es la cuenta combinada y las finas son lo que cada mercado metió **en esa misma cuenta**, así que las finas *suman* la gruesa y se leen como «cuánto del resultado es este mercado». En la pestaña Backtest, en cambio, cada mercado tiene su propia cuenta independiente y las alturas no se comparan entre sí. Un mercado cuya línea pasa toda la muestra por debajo de cero es uno al que los demás estaban sosteniendo.

No lleva cono de percentiles, y es a propósito: el remuestreo sortea bloques de calendario enteros y los concatena en el orden en que salieron, así que sus caminos no viven sobre el eje de fechas de ese gráfico. La incertidumbre está en la tabla de intervalos de debajo, no en una banda que insinuaría una línea de tiempo que no tiene.

La tabla que importa es la de **contribución marginal**: la cartera entera, la cartera sin cada mercado, y la diferencia. **Δ positivo = ese mercado mejora la combinación.** Un mercado con Δ Ret/DD negativo le está costando más de lo que aporta, por bien que se viera su p-valor.

Dos maneras de preguntar cuánta suerte hay, que no son la misma:

- **Remuestreo por bloques de calendario** (cuatro semanas por defecto): se sortean semanas enteras, así que las operaciones de todos los mercados dentro de un bloque viajan juntas y una semana mala para dos mercados a la vez sigue siéndolo. Remuestrear operaciones sueltas destruiría justo eso.
- **Barajado del orden**: las mismas operaciones en otro orden. La composición no cambia, así que el beneficio sale idéntico por construcción y sólo se mueven la caída, la racha y el Ret/DD.

Ojo con una cosa, y la pestaña lo dice: **el oro está sobreajustado**, así que la cartera base se ve mejor de lo que es.

La pestaña «Entrada aleatoria · OOS principal»

El retest en otros mercados es un OOS **de mercado**: la estrategia no se optimizó ahí. Pero el backtest principal ya traía dentro un OOS **de tiempo**, el tramo que el proyecto reservó y sobre el que el generador no optimizó nada. Esta pestaña le hace exactamente la misma pregunta:

> *Sobre 2018–2022, en oro, ¿las entradas eligieron momento mejor que el azar?*

Es el mismo test 1a, con los mismos modelos nulos, las mismas tiradas y las mismas posiciones y costes reales. Lo único que cambia es qué se le da de comer: las operaciones del oro recortadas a ese tramo, y **las velas recortadas con ellas**. Eso segundo importa tanto como lo primero: un nulo que pudiera colocar una operación en 2010 no estaría probando el OOS, estaría probando el backtest entero con menos operaciones, con la deriva y el régimen de otros años metidos dentro.

Una operación a caballo del corte se descarta, no se recorta. Recortarla le inventaría una salida que la estrategia nunca tomó. En `Strategy 24.14.35` no hay ninguna — el oro está plano en el cambio de año— así que aquí no cuesta nada; en una estrategia que aguante más, costará.

Por qué no suma con plata y Brent. No es otro mercado: es el mismo, sobre fechas que solapan con las suyas. El p conjunto de la pestaña Backtest está dimensionado para mercados distintos desplazados a la vez, así que esta fila se reporta aparte y no entra ni en él, ni en el recuento de mercados, ni en el portfolio, ni en la matriz de correlación. En la cuenta combinada el oro ya está entero.

No tiene barrido de ventana, y no es un olvido: el barrido parte la muestra en bloques de 3 años, 1 año y 6 meses, y cinco años con unos cientos de operaciones deja todos los bloques por debajo de `sweep.min_trades`. Saldrían cuatro celdas vacías. El estrés de coste y ejecución sí se corre, que no necesita ese hueco.

Lo que hay que leer antes que el p

 Ese tramo no es dato virgen. Las condiciones de aceptación del proyecto lo leyeron igualmente, dos veces: dentro de las 8 condiciones `sampleType=127`, porque el periodo completo lo contiene, y de forma explícita por el beneficio neto OOS de la matriz walk-forward. Medido sobre `Build-Task3.xml`.

Eso no invalida el número: un p bajo aquí sigue diciendo que **en ese tramo** el momento de entrada aporta por encima del azar, que es una afirmación sobre la mecánica de esa ventana y está bien calculada. Lo que no dice es que la estrategia funcione sobre datos que nadie había mirado, y sobre una población de estrategias seleccionadas está sesgado a la baja. De las 30 de `Retest Markets - Family`, **30 son rentables en oro entre 2018 y 2022**: ésa es la forma que tiene una ventana seleccionada, no la que tiene una desconocida.

 La única ventana fuera de muestra en sentido estricto de este proyecto empieza el **2023-01-01**: ningún `dateTo` de ninguna tarea pasa de `2022.12.31`, mientras que los ficheros de velas llegan a 2026. Un retest desde ahí es lo que hace falta para una afirmación limpia, y lo diría del oro y de los mercados adicionales a la vez. Está en `OPEN.md`.

El estudio dice todo esto solo: el aviso se llama `selected_window`, sale al final de la pestaña con sus cuatro partes, y es el único de los nueve que no se comprueba sino que se **adjunta** — nada en los datos puede delatarlo, es un hecho sobre el proyecto.

Los costes del estrés salen de un fichero que tienes que revisar

`studies/transfer/crossmarket/execution.yaml` lleva, por cada mercado, el spread típico y el de estrés en puntos, la comisión en dólares por lote y por lado, el slippage típico y el tamaño del punto. De ahí salen el multiplicador de coste y la profundidad de fill del estrés, en vez de los números redondos que había antes.

⚠ **Los valores de hoy son de ejemplo**, escritos como los de un bróker CFD normal para que tengas algo concreto que corregir. Cada bloque lleva `source: placeholder` y `reviewed_by_owner: false`, y la tabla lo dice en la columna «origen». Cuando pongas los tuyos, cambia esa marca a `true`.

No están en `assets/*.yaml` a propósito: esos ficheros bloquean la creación de proyectos en todo el repositorio y `assets/RULES.md` dice que un fichero con valores inventados es peor que no tener fichero, porque parece decidido.

Cómo se leen los dos gráficos nuevos

El cono de equity. La línea naranja es el backtest real; la banda azul es donde corrieron los aleatorios. El eje X es **tiempo de calendario**, no número de operación: las entradas aleatorias caen en momentos distintos, así que sólo en ese eje la curva real y las suyas describen el mismo tramo de mercado. Se lee por el **ancho** del cono y por **dónde** la real se sale de él, nunca por una línea suelta de dentro.

Los histogramas. Las barras son los backtests aleatorios, la línea naranja el real, la discontinua la mediana simulada y el sombreado la banda del 5 al 95. Ojo con la dirección: en **drawdown** y **racha perdedora**, menos es mejor, así que un p pequeño significa que el real sufrió **menos** que el azar. Cada fila de la tabla lo dice al lado.

Todos los backtests aleatorios se valoran **en dólares, con los mismos tamaños de posición y los mismos costes** que las operaciones reales. Por eso el beneficio neto que ves es directamente el de SQX: el P/L reconstruido desde las velas correlaciona 0,9996 con el que reporta la databank.

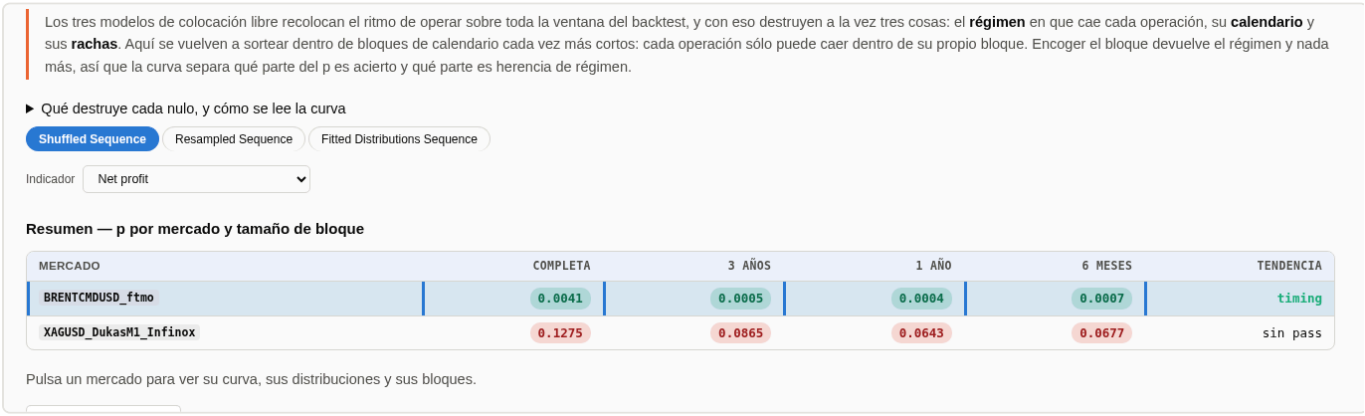
El barrido de ventana: ¿acierto o suerte de régimen?

Shuffled, Resampled y Fitted Sequence colocan tu ritmo de operar en cualquier punto de la ventana del backtest. Con eso rompen a la vez tres cosas: **en qué régimen** cae cada operación (una tendencia de 2011 o un lateral de 2019), **su calendario** (día, hora, fines de semana) y **sus rachas**. Si uno de ellos da un p bajo, no sabes si es porque tus entradas aciertan o porque tus operaciones cayeron por suerte en tramos buenos.

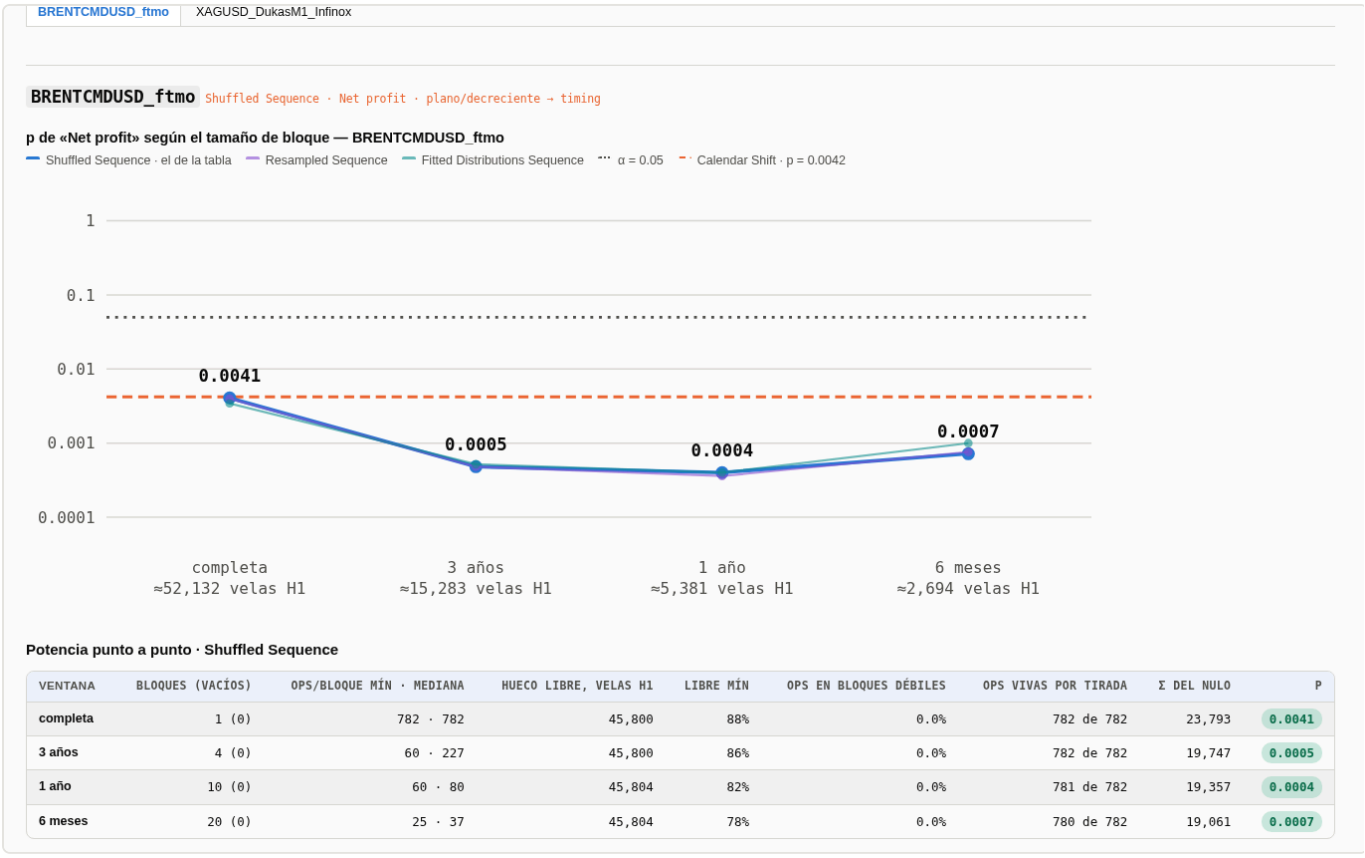
El barrido lo separa. Repite esos tres modelos, pero obligando a cada operación a quedarse dentro de un bloque de calendario: primero la ventana entera, luego bloques de 3 años, de 1 año y de 6 meses. Cuanto más pequeño el bloque, más cerca de su fecha real cae cada operación, así que **más régimen se le devuelve** — y es lo único que cambia: el calendario y las rachas siguen rotos igual a todos los tamaños.

Cómo se navega la pestaña. Lo primero que ves es una rejilla: una fila por mercado, una columna por tamaño de bloque, el p de cada celda coloreado y la tendencia a la derecha. Ahí está el barrido entero sin bajar nada, y con varios mercados es donde se compara. Los **chips de arriba** eligen el modelo de colocación

libre y el desplegable **Indicador** el estadístico —abre en **Net profit**—: entre los dos gobiernan toda la pestaña, rejilla incluida. Cada punto del barrido es ya un nulo completo, así que el p, la tendencia, los histogramas y el cono se recalculan para el indicador que elijas sin volver a correr nada.



Pulsando una fila —o usando las sub-pestañas de mercado— se abre ese mercado debajo: **una sola curva con los tres modelos**, el seleccionado en trazo grueso y con sus valores escritos, los otros dos detrás para ver si coinciden. Debajo, la tabla de potencia del modelo seleccionado, y al final los bloques, plegados.



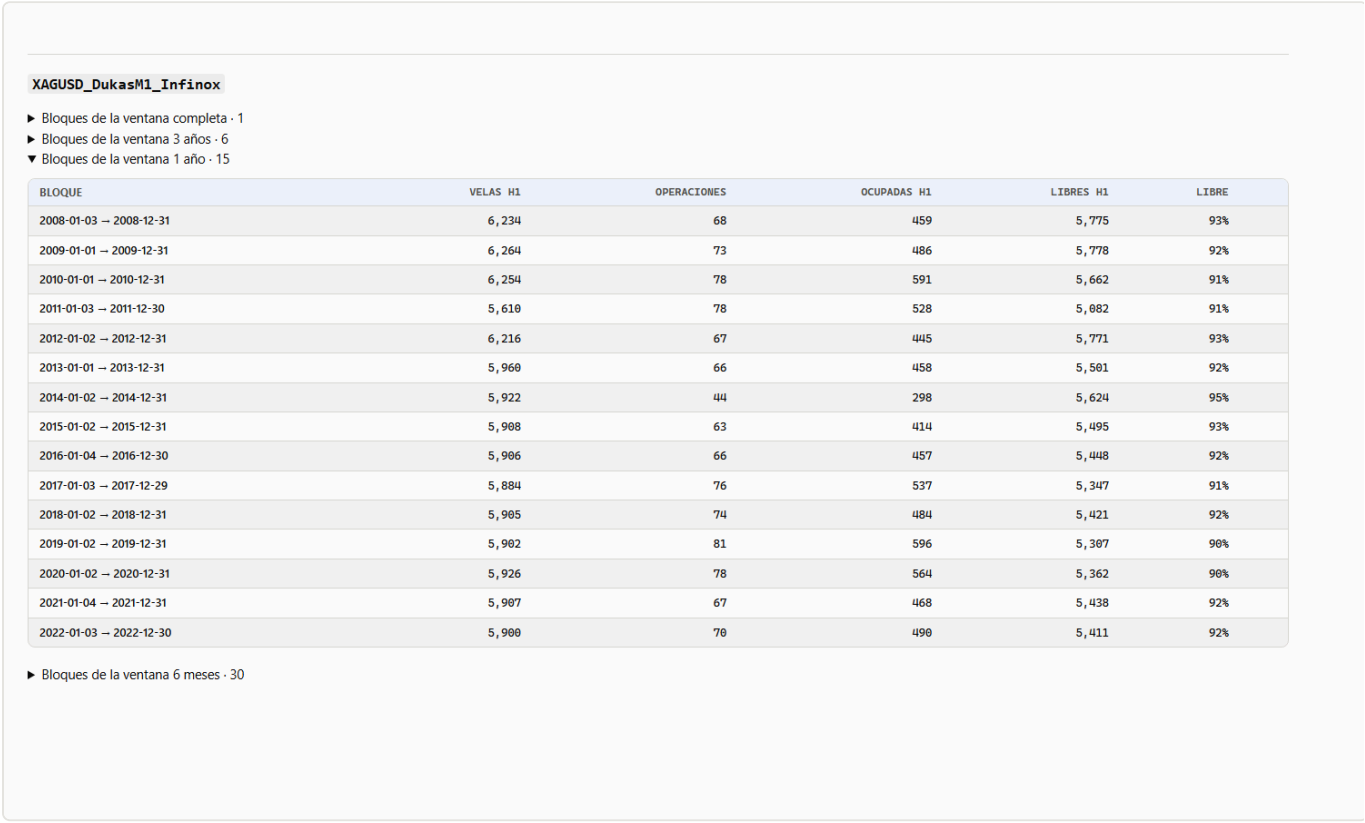
Cómo se lee la curva:

- **p se mantiene bajo al encoger el bloque** → el acierto sobrevive aunque le quites la suerte de régimen. Es timing.
- **p sube al encoger el bloque** → el aprobado con la ventana entera era herencia de régimen.

- La **línea naranja discontinua** es el p de Calendar Shift, fijo. **La curva nunca va a llegar a ella**: Calendar Shift conserva también calendario y rachas, que el barrido sigue rompiendo. Es otra cosa; está ahí como referencia.
- La **línea punteada** es alpha. El eje es logarítmico: cada raya es un orden de magnitud.

Al lado del nombre del mercado hay una etiqueta automática, y la rejilla la repite en su última columna: **plano/decreciente → timing**, **creciente → régimen** (p sube más de un orden de magnitud del bloque más ancho al más estrecho), **sin pass a ningún tamaño** (ningún punto llega a alpha, así que no hay aprobado que descomponer) o **no evaluable**. Es un pie de foto, no la lectura.

Nunca leas el p solo. Con bloques pequeños hay menos sitio donde recolocar, el nulo se ensancha y p pierde resolución. Por eso la tabla de debajo del gráfico da, para cada tamaño, cuántos bloques hay, cuántas operaciones tiene cada uno, cuánto hueco libre queda, qué parte de las operaciones cae en bloques débiles, cuántas operaciones sobreviven por tirada y la σ del nulo. Si demasiadas operaciones caen en bloques con menos de 10 operaciones o con menos de un 25% de velas libres, ese tamaño **no se calcula** y sale como ×. Debajo de todo, desplegable, la lista de bloques de cada tamaño:



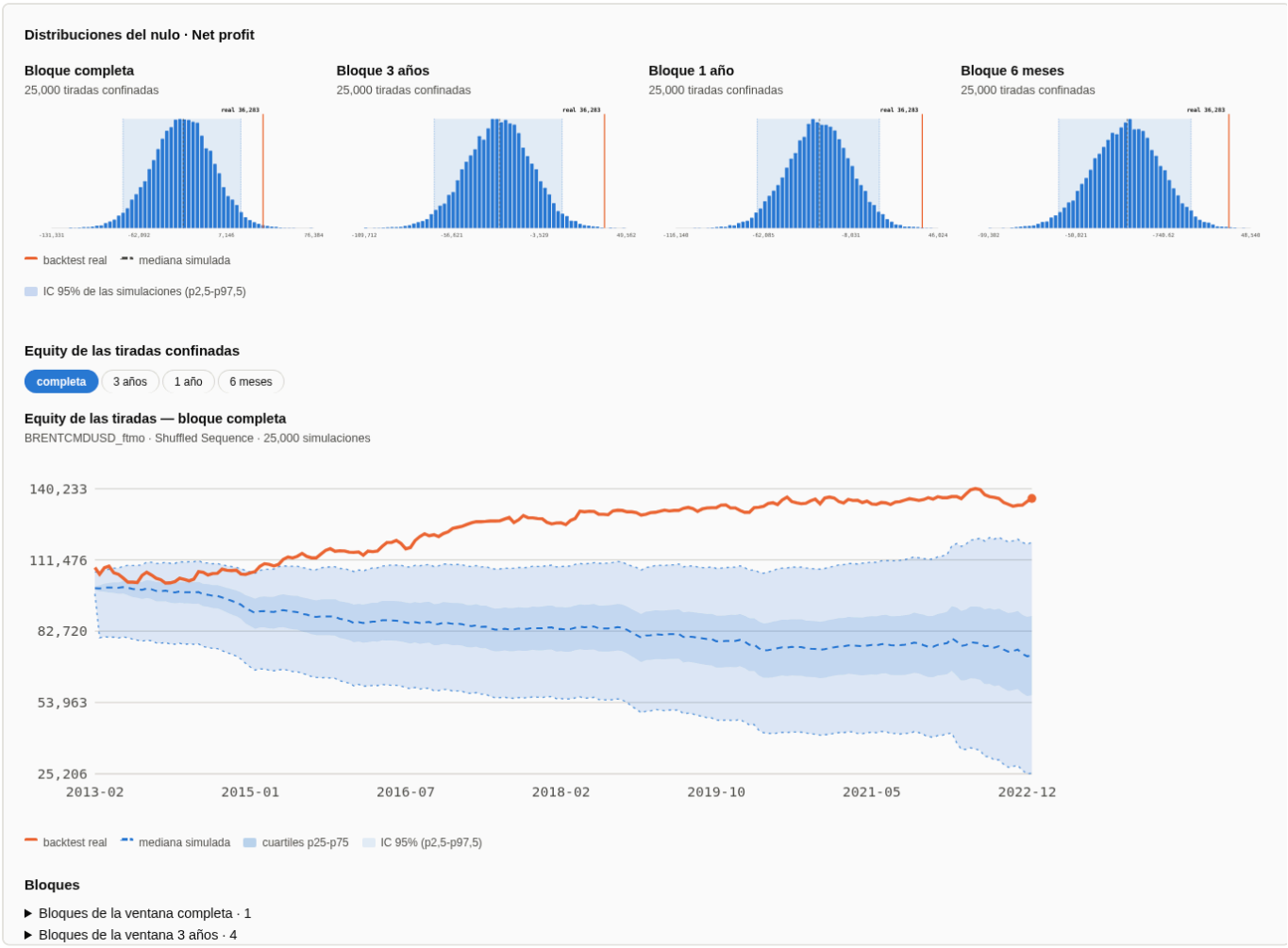
`XAGUSD_DukasM1_Infinox`

▸ Bloques de la ventana completa · 1
▸ Bloques de la ventana 3 años · 6
▾ Bloques de la ventana 1 año · 15

BLOQUE	VELAS H1	OPERACIONES	OCUPADAS H1	LIBRES H1	LIBRE
2008-01-03 → 2008-12-31	6,234	68	459	5,775	93%
2009-01-01 → 2009-12-31	6,264	73	486	5,778	92%
2010-01-01 → 2010-12-31	6,254	78	591	5,662	91%
2011-01-03 → 2011-12-30	5,610	78	528	5,082	91%
2012-01-02 → 2012-12-31	6,216	67	445	5,771	93%
2013-01-01 → 2013-12-31	5,960	66	458	5,501	92%
2014-01-02 → 2014-12-31	5,922	44	298	5,624	95%
2015-01-02 → 2015-12-31	5,908	63	414	5,495	93%
2016-01-04 → 2016-12-30	5,906	66	457	5,448	92%
2017-01-03 → 2017-12-29	5,884	76	537	5,347	91%
2018-01-02 → 2018-12-31	5,905	74	484	5,421	92%
2019-01-02 → 2019-12-31	5,902	81	596	5,307	90%
2020-01-02 → 2020-12-31	5,926	78	564	5,362	90%
2021-01-04 → 2021-12-31	5,907	67	468	5,438	92%
2022-01-03 → 2022-12-30	5,900	70	490	5,411	92%

▸ Bloques de la ventana 6 meses · 30

Las distribuciones y el cono. Debajo de la tabla hay un histograma por tamaño de bloque, los cuatro en fila y con el backtest real marcado en cada uno: ahí se ve —no se argumenta— cómo el nulo se ensancha o se estrecha al encoger el bloque, que es de dónde sale la pérdida de resolución del p. Y debajo, el **cono de equity** de las tiradas confinadas del tamaño que elijas con los chips, con la curva real encima, igual que en «Entrada aleatoria» pero con las operaciones atadas a su bloque.



Un aviso sobre la columna de operaciones vivas: en Resampled y Fitted Sequence baja un poco al encoger el bloque (en plata, de 98% a 94% entre la ventana entera y 6 meses), porque lo que no cabe en un bloque se descarta. Shuffled Sequence no tiene ese efecto.

Medido sobre 6 estrategias, 2 mercados y los 3 modelos, a 10.000 tiradas:

estrategia	mercado	modelo	completa	3y	1y	6m	etiqueta
Strategy 2.29.29	XAGUSD	segment_permute	0.0017	0.0012	0.0013	0.0023	timing
Strategy 1.10.80	XAGUSD	segment_permute	0.0507	0.0463	0.0482	0.0421	timing
Strategy 14.15.26(2)	BRENT	segment_permute	0.0752	0.0873	0.0953	0.1167	sin pass
Strategy 14.15.26(2)	XAGUSD	segment_permute	0.1048	0.1211	0.1321	0.1387	sin pass
Strategy 15.17.41	XAGUSD	segment_permute	0.1924	0.1974	0.1819	×	sin pass

La primera es un acierto que aguanta a todos los tamaños. La tercera y la cuarta tienen justo la forma del régimen — p sube al encoger — pero nunca bajaron de alpha. La última no tiene potencia a 6 meses: el 13% de sus operaciones cae en bloques débiles.

Regime Strata, apagado por defecto, es otra forma de hacer lo mismo sin elegir «un año»: recoloca cada operación en una vela del mismo tipo de mercado (cuantil de volatilidad y signo de la tendencia reciente). Para activarlo, añade `regime_strata` a `nulls.models` en el cajón; aparece como un modelo más en Entrada aleatoria y Modelos, no dentro del barrido.

Cómo se lee el resultado

El p conjunto: ¿se traslada, o acertó en un mercado?

Un p por mercado no contesta la pregunta del retest. Si miras cuatro mercados con alpha en 0,05, que **uno** baje de 0,05 no dice casi nada — y contar cuántos bajan tampoco vale, porque esos p-valores **no son independientes**: salen de los mismos sorteos y de mercados que se mueven juntos. Combinarlos con Fisher, con Stouffer o por votación supone independencia justo donde no la hay, y el resultado sale más generoso de lo que debería.

La pestaña Backtest trae por eso **un solo número decidido de antemano**: la media de `mean_r` sobre los mercados fuera de muestra —un voto por mercado, el activo base fuera— medida contra esa misma media en cada sorteo. Y lo que lo hace correcto no es la media, es que **el mismo desplazamiento de calendario se aplica a todos los mercados a la vez**: si en el sorteo 412 el primer semestre de 2013 se mueve nueve semanas, se mueve nueve semanas en Brent y en plata. Así el remuestreo ya lleva dentro la correlación que haya entre los mercados y no hay que modelar ninguna matriz.



Si lo ves **sin número**, es por una de tres razones y la pestaña la dice: sólo hay un mercado fuera de muestra, los mercados se corrieron con distinto número de tiradas (te pasará si re-ejecutas uno solo cambiando `nulls.draws`), o esta sesión no guardó los sorteos.

El desplegable `joint.pool` del cajón decide cómo se agregan los mercados. Por defecto, `mean_r` crudo, que es lo correcto mientras las anchuras del nulo de cada mercado estén parecidas —medido, 0,1236 en Brent contra 0,1026 en plata, un factor de 1,20—. Cámbialo a `z` si algún día entra un mercado cuyo nulo sea varias veces más ancho, porque entonces decidiría él solo.

Por dónde salieron las operaciones

Estas estrategias salen por tres sitios: **tope de barras**, **cierre del viernes** y **señal**. Los dos primeros el nulo los reproduce —uno cuenta barras, el otro mira el calendario—; el tercero no, porque haría falta leer el `.sqx`. Por eso el p de una estrategia con salidas por señal es un test conjunto de entrada **y** salida.

Lo que la pestaña añade ahora es **cuánto dinero** hay en cada tipo de salida, no sólo cuántas operaciones. No es lo mismo: medido en `Strategy 24.14.35`, la salida por señal es el 16,5% de las operaciones en Brent pero sólo el **6,2% del P/L bruto**, y encima pierde dinero (−13.546 $). El **93,8% del resultado** descansa en salidas que el nulo sí reproduce, así que en ese mercado la advertencia de «entrada + salida» cubre una esquina pequeña del resultado. En otra estrategia podría ser al revés, y entonces el p diría mucho menos de lo que parece.

`BRENTCMDUSD_ftmo`

SALIDA	OPERACIONES	% DE OPERACIONES	P/L NETO	% DEL P/L BRUTO	¿LA REPRODUCE EL NULO?
Exit After X Bars	626	80.1%	+48,437 $	89.3%	sí
Exit Signal	129	16.5%	-13,546 $	6.2%	no
End Of Friday (Time)	27	3.5%	+1,392 $	4.4%	sí

93.8% del P/L bruto de este mercado sale de operaciones cuya salida el nulo sí reproduce — tope de barras y cierre de viernes. El resto descansa en la salida por señal, que no se puede reproducir sin leer el `.sqx`: para esa parte el p es un test conjunto de entrada **y** salida.

`XAGUSD_DukasM1_Infinox`

SALIDA	OPERACIONES	% DE OPERACIONES	P/L NETO	% DEL P/L BRUTO	¿LA REPRODUCE EL NULO?
Exit After X Bars	663	78.6%	+24,226 $	87.8%	sí
Exit Signal	154	18.2%	-16,593 $	8.0%	no
End Of Friday (Time)	27	3.2%	-3,705 $	4.3%	sí

92.0% del P/L bruto de este mercado sale de operaciones cuya salida el nulo sí reproduce — tope de barras y cierre de viernes. El resto descansa en la salida por señal, que no se puede reproducir sin leer el `.sqx`: para esa parte el p es un test conjunto de entrada **y** salida.

La z, al lado del p

En la tabla «Qué dijo cada test» hay ahora una columna **z (1a)** junto al p. Existe por dos motivos: el p **satura** —con 25.000 tiradas el más pequeño posible es 0,00004, así que dos mercados excelentes salen iguales y no lo son—, y el nulo es **más ancho en el mercado con menos operaciones**, lo cual no es edge. La z divide esa anchura: es (real − media del nulo) en desviaciones típicas del nulo. Sirve para **comparar mercados y ordenarlos**, no para decidir: el p exacto sigue siendo el test, y la z **no se convierte en un p** en ninguna parte, porque esa conversión no vale para el drawdown ni para la racha perdedora.

El orden en que hay que leer el estudio de una estrategia:

1. **Las comprobaciones.** `fill_error` tiene que estar muy por debajo de `max_fill_error` (0,25 ATR) y `calendar_kept` valer 1,00 en `block_shift`. Si el primero se dispara, las velas no son las del backtest y no hay nada que interpretar: para ahí. Ojo con `fill_error`: **no tiene que ser 0**. Lleva dentro el spread de entrada, que es una constante que paga también cada corrida aleatoria —el oro marca 0,023 ATR—; lo que delata un problema es que sea grande, no que exista. El spread sale desglosado en su propia fila.
2. **Los avisos**, que ahora están dentro de la propia pestaña Backtest y también en la suya. Dicen de qué desconfiar en cada mercado antes de que te enamores de un número — y, sobre todo, **a qué NO afecta** cada uno: casi ninguno invalida la fila entera.
3. **La tabla maestra** de la pestaña Backtest: lo que hizo el backtest de verdad, antes de ninguna simulación. Y al lado, la de riesgo igualado.
4. **Lo que dijo cada test**, en la misma pestaña — el p de cada mercado con su z al lado.
5. **El p conjunto**, que es el que contesta si se traslada.
6. **Sólo entonces**, el cono y los histogramas de cada pestaña.

Y una cosa que no cambia por tener mejores gráficos: estás mirando **una estrategia de 757**, que además ya pasaron por la búsqueda de SQX. Con alpha en 0,05, de 757 estrategias unas 38 darían un p ≤ 0,05 en un mercado por puro azar. Un resultado bonito aquí no es un descubrimiento hasta que sepas contra cuántos intentos lo estás comparando.

Las columnas de la tabla por mercado que hay que saber leer:

columna	qué es	cómo se lee
`real_r`	lo que ganó de verdad por operación, en "barras típicas de ese mercado"	**no significa nada solo**
`null_r`	lo que ganó la versión aleatoria mediana	es la vara de medir; suele rondar cero
`edge_r`	la diferencia entre las dos	es el tamaño del efecto, y es lo que hay que mirar cuando el p sale ajustado
`p`	qué fracción de las versiones al azar igualó o superó a la real, bajo el primer modelo	es el número principal del test 1a
`p_<modelo>`	lo mismo bajo cada forma alternativa de aleatorizar	comprobación, no segunda opinión
`paired_p`	el p del test pareado (1b)	**no depende de ningún modelo nulo ni de ninguna suposición de coste**
`paired_beat`	qué porcentaje de operaciones batió a su ventana ciega	50% es el azar
`a` / `a_ci_lo` / `a_ci_hi`	el exceso por vela sobre la vela media del mercado, con su intervalo	si el intervalo cruza el cero, el exceso no está demostrado
`e` / `e_ci`	la concentración E y su intervalo de Fieller	cuando el intervalo sale **«no acotado»**, E no está determinada: mira A por unidad de riesgo
`risk_normalised`	A dividida por el movimiento típico de una vela de ese mercado	**es el número principal de 1c**: está definido en todos los mercados
`paired_usd_total`	el alfa de timing acumulado en dólares sobre toda la muestra	cuánto del dinero que ganó lo puso el **momento** de entrar
`mu_t`	el t de la deriva del propio mercado	por debajo de 2 en valor absoluto, ese mercado no tiene tendencia que dividir
`breakeven`	a cuántas veces el coste real deja de ganar	el PDF pide 2,0 o más
`trades_all`	**todas** las operaciones que reporta SQX	es con las que cuadran el beneficio, la caída y el PF de la pestaña Backtest
`trades` / `off_grid`	las que ocupan al menos una vela, y las que no	las que no son, casi siempre, salidas `Exit Signal` de duración `0s`: son reales y su dinero cuenta, pero 1a, 1b y 1c necesitan una duración y no pueden usarlas
`on_open_price`	fracción de entradas ejecutadas **al precio** de su propia vela, descontado el spread constante	por debajo de 0,95 hay fills intravela que el azar no puede colocar: dispara aviso, **no** excluye. Si cae mucho (< 0,7) sospecha del timeframe antes que de las órdenes
`on_bar_open`	fracción de entradas cuyo **reloj** cae en el inicio de barra	informativo y nada más. □ No dispara nada: de 960.705 operaciones, las 4.613 selladas a mitad de vela están al mismo precio que el resto. Que sea 0,91 mientras `on_open_price` es 1,00 es normal

columna	qué es	cómo se lee
`fill_error`	error mediano de precio contra las velas, en múltiplos del ATR	por encima de `max_fill_error` las velas no son las del backtest
`fill_offset`	el spread de entrada, en múltiplos del ATR	constante por mercado; lo paga también cada corrida aleatoria porque el coste se recupera operación a operación
`calendar_kept`	fracción de entradas aleatorias en el mismo día y hora que la real	1,00 en `block_shift`; vacío en `renewal`, que no empareja operaciones
`null_trades`	operaciones por backtest aleatorio	igual a las reales salvo en `renewal`
`atr_ratio`	volatilidad en las entradas reales frente a la media del mercado	cerca de 1. Lejos de 1 significa que la estrategia elige barras raras
`warnings`	la lista de motivos para desconfiar de esa fila	está para leerse, no para filtrar
`net` / `dd` / `ret_dd`	beneficio neto, drawdown máximo y su cociente, en dólares	son los de SQX: el P/L reconstruido correlaciona 0,9996 con el suyo
`p_net` / `p_dd`	dónde cae cada uno dentro de los backtests aleatorios	en `p_dd`, pequeño es **bueno**: el real aguantó mejor que el azar

Sobre `family`, en la pestaña Backtest. Cuando la estrategia sale siempre por el tope de barras, la duración no dependía del precio y esto es una prueba limpia de la entrada. Cuando sale por señal, la duración sí lleva información y el nulo la reutiliza sin poder reproducir de dónde salía: para esas, el resultado mide entrada **y** salida a la vez. No es peor, es otra cosa.

Dos avisos que vas a ver mucho y qué significan de verdad:

- `no_drift` — ese mercado no tiene una deriva distinguible de cero en la ventana medida. Medido: oro t = +3,13, plata t = +1,61, Brent t = −0,11. Afecta **sólo a E**, que divide por esa deriva: su intervalo de Fieller sale entonces «no acotado», que es la verdad. **A no se ve afectada** — resta en vez de dividir — y de hecho Brent, que dispara este aviso, tiene la A más alta de los tres.
- `short_sample` — el Sharpe por operación observado necesitaría más operaciones de las que hay para distinguirse de cero (MinTRL). Es casi universal con Sharpes por operación de 0,03-0,15: dice que la evidencia de *rentabilidad* es débil, no que el test de *timing* esté mal.

Un ejemplo completo

Prueba del módulo sobre el propio oro (2007-2018, 5000 versiones al azar por estrategia). Recuerda que aquí se espera que gane: es el mercado en el que se optimizaron.

```
8 estrategias en 3.9s (5000 tiradas)

estrategia        n   real_r   null_r    edge  p_shift   p_perm    cal   atr  barcap
1.17.44.csv    1101    0.156   -0.013   0.169   0.0006   0.0010   1.00  1.02    0.48
1.25.34.csv    1038    0.129   -0.003   0.132   0.0072   0.0052   1.00  1.10    0.59
10.15.25.cs    1144    0.133   -0.018   0.151   0.0004   0.0022   1.00  1.03    1.00
14.16.34.cs    1035    0.236   -0.033   0.269   0.0002   0.0004   1.00  1.03    1.00
15.24.36.cs    1230    0.184   -0.010   0.194   0.0002   0.0004   1.00  1.08    0.40
16.20.43.cs     933    0.205   -0.006   0.211   0.0002   0.0002   1.00  1.05    0.17
```

Y la comprobación de que el número mide lo que dice medir: se cogen las operaciones de una estrategia y se retrasan todas unas cuantas barras, sin tocar nada más. Si el módulo mide acierto en el momento de entrar, moverlas tiene que estropearlo — y lo hace, de forma ordenada:

```
 desplazo   real_r     edge   p_shift
        0    0.236    0.269    0.0002     <- la real
        1    0.207    0.226    0.0010     <- una hora tarde
        3    0.120    0.120    0.0240     <- tres horas tarde: ya casi no significa nada
        8    0.064    0.072    0.1580     <- ocho horas tarde: indistinguible del azar
       24    0.065    0.098    0.1262
```

Qué NO te dice

- **No te dice que la estrategia esté sobreajustada o no al activo base.** Mide traslado, no ajuste.
- **La pestaña Portfolio no construye tu cartera real.** Es una estrategia en N mercados, para ver si uno de ellos rompe la combinación. Tu cartera real, con varias estrategias, es otro módulo.
- **No valida la curva de capital.** El número está construido a propósito sin tamaño de posición, para que la comparación sea justa. Dice que entra en barras mejores que el azar; no dice que la gestión monetaria funcione.
- **No convierte un mercado fallado en un problema.** Que falle en un mercado es información sobre dónde vive la ventaja, y es la materia prima del estudio de propiedades de mercado.
- **No corrige por haber probado cientos de estrategias a la vez.** Declara cuántas saldrían así por suerte; no las quita. Ese es el número que tienes que mirar tú.
- **No decide nada.** No hay veredicto, no hay lista de supervivientes y no hay ningún mercado escondido. Es una herramienta de medida; la decisión es tuya.

Si algo falla

- **Un tamaño del barrido sale como ✕** — no es un error: ese tamaño no tiene potencia para esa estrategia en ese mercado. La tabla dice por qué. Si quieres verlo igualmente, baja `sweep.min_trades` o sube `sweep.max_weak_share` en el cajón, y léelo sabiendo lo que has hecho.
- **`FileNotFoundError` sobre un fichero de `bars/`** — ese mercado no está exportado. Ejecuta el paso 1, o quítalo de `assets/_markets.yaml`.
- **`KeyError` con el nombre del activo** — el activo no tiene bloque en `assets/_markets.yaml`.
- **Un mercado sale como `sin clasificar`** — está en el export pero no en `assets/_markets.yaml`. Se analiza igual; añádelo a la categoría que le toque cuando quieras que salga etiquetado.

- **Un mercado declarado sale como ausente al arrancar** — está en `assets/_markets.yaml` pero el export no trae operaciones suyas. O el nombre del feed no coincide con el que usó la tarea de retest, o la estrategia no operó ahí.
- **`ValueError: cross-market pricing is long-only`** — alguna estrategia lleva operaciones en corto. Todo el retorno de este módulo es `log(salida/entrada)`, que para un corto tiene el signo al revés, así que se niega a valorarlo en vez de dar un número equivocado.
- **`fill_error` por encima de `max_fill_error`** — las barras y las operaciones no son del mismo mercado o de la misma ventana. El resultado no vale; no lo interpretes. Que `fill_error` sea distinto de 0 y pequeño **no** es esto: es el spread de entrada, y es normal.
- **`on_open_price` muy por debajo de 0,95** — o hay órdenes limit/stop llenadas dentro de la vela, o has cargado el timeframe equivocado. Probado: velas H1 para un backtest M30 dejan este número en 0,57 sin tocar `fill_error`.

39. Crossmarket por lotes — el paso 10 sin abrir el panel

Qué pregunta responde

Si una población entera de estrategias sigue ganando en los mercados que **nunca vio**, y escribe la respuesta como una lista de supervivientes que el paso siguiente de SQX puede leer. Es la mitad automática del paso 10 del workflow: en la ventana, la pestaña **Cross Market** del panel de databanks (zona Proyecto) y la ficha de la estrategia (zona Estrategia) siguen siendo donde se mira **una** estrategia con lupa; esto es para cribar las ochenta.

La pregunta concreta que contesta es **amplitud**: en cuántos de los mercados declarados el intervalo de confianza de la esperanza por operación queda por encima de cero. Se eligió ésa y no un p-valor porque la amplitud no necesita creerse ningún modelo: que la esperanza de una estrategia supere cero en un mercado al que nunca se ajustó es aritmética. Los p de los modelos de colocación salen en el CSV al lado, y **no deciden nada**.

Cuándo lo usas, y cuándo no

- **Sí**: después de correr la tarea `Retest Markets - Family` de tu proyecto y exportar sus operaciones, cuando quieres que el paso 13 (MC Retest) sólo vea a los que transfieren.
- **No**: para entender *por qué* una estrategia falló en un mercado. Eso es la ficha de la estrategia en la ventana (zona Estrategia, pestaña Transferencia › Cross-market), que dibuja la curva, la matriz de correlación y la cartera combinada. Un CSV no te dice eso.
- **No** como única lectura de un edge: el veredicto aquí es un suelo, no un dictamen.

⚠ **Esto guarda un resultado en disco.** El informe por lotes se había quitado en 2026-09-15 precisamente por eso —un número guardado se puede leer como respuesta a una pregunta para la que no se calculó— y volvió el 2026-09-24 a petición del dueño, para que la cadena corra sin humano. El propio comando te lo recuerda al terminar.

Antes de empezar

Tres cosas, y las tres fallan de forma clara si faltan:

1. **La tarea corrida y exportada.** `python3 -m sqx.export.export_retest --project <P> --databank "Retest Markets - Family" --role custodian` deja el `trades.parquet` que esto lee.
2. **Las barras M1 de TODOS los mercados de la familia** en la librería, porque cada celda se puntúa contra su propio mercado: `python3 -m sqx.export.sync_bars --check` te dice qué falta y sin el flag las trae. Son ~8,6 M de barras y ~140 MB por par.
3. **El activo declarado** en `assets/_markets.yaml`, que es de donde sale la lista de mercados.

No toca SQX, así que puedes lanzarlo con la GUI del maestro abierta.

Cómo se ejecuta

```
python3 -u -m studies.transfer.crossmarket.report \
    --project TestUSDJPY_Workflow_v1 \
    --databank Retest_Markets_-_Family \
    --asset USDJPY \
    --day 2026-09-24 \
    --set nulls.batch_draws=2000
```

flag	obligatorio	qué hace
`--project`	sí	el proyecto tal y como aparece en SQX
`--databank`	sí	la carpeta de la exportación, **con guiones bajos** donde el databank lleva espacios
`--asset`	sí	el activo base, p. ej. `USDJPY`. Decide la lista de mercados
`--day`	no	la fecha de la exportación, `AAAA-MM-DD`; hoy si no se pone
`--floor`	no	fracción de mercados que tienen que superar cero para conservarla. Por defecto `0.5`
`--set`	no	cualquier knob de `config.yaml`, p. ej. `nulls.batch_draws=2000` para una prueba rápida
`--workers`	no	cuántos **mercados** se estudian a la vez. Por defecto los 48 núcleos físicos, que es también el máximo: con más iba más lento

Cada tarea es **una estrategia en un mercado**, no una estrategia entera, y las más largas salen primero, repartidas en los 48 núcleos físicos.

⚠ **Pon el `-u` de Python.** Y no lo canalices a `tail`: una tubería retiene la salida hasta el final y un lote de una hora se vuelve indistinguible de un cuelgue.

Lo que tarda y lo que ocupa. Medido el 2026-09-26 con la configuración por defecto (10.000 sorteos en el lote, `nulls.batch_draws`):

población	operaciones	tiempo	memoria (todo el árbol de procesos)
8 estrategias × 9 mercados	124.000	**10 s**	4,3 GB
499 estrategias × 9 mercados	12,0 M	**6 min 16 s**	13,1 GB

El coste va **por operación**, no por estrategia: unas estrategias llevan muchas más operaciones que otras. Antes de lanzarlo, cuenta las operaciones:

```
python3 -c "import pandas as pd; f=pd.read_parquet('.../trades.parquet'); print(len(f))"
```

Por qué 10.000 sorteos y no 25.000. El veredicto del lote mira solo el intervalo de la esperanza en cada mercado, y ese intervalo no depende del nulo. El p del nulo se sigue calculando y publicando al lado, pero no decide nada, y a 25.000 sorteos se llevaba el 84 % del cálculo. El estudio completo de una estrategia (`--strategy`) sigue usando los 25.000 de `nulls.draws`. Bájalo a 2.000 con `--set nulls.batch_draws=2000` para probar la cadena.

Qué produce

Un fichero, y **sobrescribe** el de la misma exportación:

```
~/Desktop/AlgoData/reports/<PROYECTO>/<DATABANK>/<FECHA>/crossmarket/verdict.csv
```

Catorce columnas. Las tres primeras son el contrato con `/curate` (`strategy`, `verdict`, `reason`); las once siguientes son para que no tengas que creerte el veredicto a ciegas.

Cómo se lee el resultado

Salida real de la corrida del 2026-09-24 sobre `TestUSDJPY_Workflow_v1`:

```
8 estrategias x 9 mercados a 2,000 sorteos — minutos por estrategia.
  [1/8] Strategy 11.1.50    DESCARTAR solo 0 de 9 mercados con la esperanza por encima de cero
(0% < 50%)
  ...
  [8/8] Strategy 6.1.81     DESCARTAR solo 0 de 9 mercados con la esperanza por encima de cero
(0% < 50%)

0 de 8 pasan el suelo de amplitud -> .../crossmarket/verdict.csv
```

Y la tabla, recortada a lo que se mira:

strategy	verdict	cleared/markets	under_alpha	edge_r	worst_pf	pf_cv	family
Strategy 11.1.50	DESCARTAR	0 / 9	0	−0,059	0,907	0,068	entry+exit
Strategy 23.1.71	DESCARTAR	1 / 9	2	0,046	0,982	0,045	entry+exit
Strategy 23.1.60	DESCARTAR	0 / 9	0	0,076	0,893	0,063	entry+exit

Cómo se lee cada columna:

- **`cleared` / `markets`** — la que decide. Mercados cuyo intervalo de la esperanza queda por encima de cero, sobre los que tienen operaciones. `0 de 9` es una respuesta, no un fallo.
- **`under_alpha`** — cuántos mercados dieron p ≤ 0,05 en el modelo de colocación. **Informativo.** Un 2 con `cleared` 1 dice que el p viene de un modelo y la esperanza no lo acompaña.
- **`edge_r`** — el efecto mediano en unidades del propio rango del mercado. Signo primero: en negativo la estrategia pierde en la mediana de los mercados ajenos.
- **`worst_pf`** — el profit factor del peor mercado. **Por debajo de 1 es que ahí pierde dinero.** Un buen candidato tiene el peor mercado cerca de 1, no una media alta con un desastre dentro.
- **`pf_cv`** — dispersión del profit factor entre mercados. Alto = funciona en uno y en los demás no.
- **`family`** — `entry` si todas las salidas son el tope de barras, `entry+exit` si no. Con `entry+exit` **esto no es una prueba del *timing* de entrada**, es una prueba conjunta, y así hay que contarlo.
- **`missing`** — mercados en los que la estrategia no disparó ni una vez. Es un resultado sobre la estrategia, no un dato que falte. Si no disparó en **ninguno**, su fila lo dice y no se calcula nada más: no hay nada que juzgar. (Hasta el 2026-09-24 ese caso tumbaba el lote entero con un `KeyError: 'bar_cap'`, y salió con la novena estrategia real que se probó.)

- **`warnings`** — avisos que la estrategia acumuló entre mercados. 25-27 sobre nueve mercados es normal cuando los costes son provisionales: casi todos son ése.

Aplicarlo

```
python3 -m sqx.curate.apply_verdict --project <P> --databank "Retest Markets - Family" \
    --verdict .../crossmarket/verdict.csv --role custodian          # mira primero
python3 -m sqx.curate.apply_verdict ... --apply                    # con el custodio parado
```

Un ejemplo completo

```
# 1. exporta las operaciones de la tarea, con el custodio parado
bin/sqx-worker.sh --role custodian stop
python3 -m sqx.export.export_retest --project TestUSDJPY_Workflow_v1 \
    --databank "Retest Markets - Family" --role custodian

# 2. asegúrate de tener las barras de los nueve pares
python3 -m sqx.export.sync_bars --check

# 3. juzga la población entera
python3 -u -m studies.transfer.crossmarket.report --project TestUSDJPY_Workflow_v1 \
    --databank Retest_Markets_-_Family --asset USDJPY --day 2026-09-24 \
    --set nulls.batch_draws=2000

# 4. aplica el veredicto para que el MC Retest sólo vea a los supervivientes
python3 -m sqx.curate.apply_verdict --project TestUSDJPY_Workflow_v1 \
    --databank "Retest Markets - Family" --role custodian \
    --verdict ~/Desktop/AlgoData/reports/TestUSDJPY_Workflow_v1/Retest_Markets_-_Family/2026-
09-24/crossmarket/verdict.csv
```

Qué NO te dice

- **No dice que el edge sea real.** Dice que sobrevive a cambiar de mercado, que es una condición necesaria y nada más. Un mono largo en nueve pares correlacionados también puede pasarlo.
- **No corrige por multiplicidad entre estrategias.** Cribar ochenta estrategias con este suelo es ochenta pruebas, y el CSV no lo paga. Eso es el ledger global (`03-datos-costes-y-registro.pdf › 43-ledger`).
- **No es una prueba del timing de entrada** cuando `family` dice `entry+exit`.
- **No sustituye a la ficha de la estrategia.** Sin la curva y la matriz de correlación no se ve si los nueve mercados son nueve observaciones o una repetida nueve veces.
- **Con costes provisionales, no juzga nada de nivel.** Mira la columna `warnings`.

Si algo falla

- **`KeyError: 'AUDJPY_DukasM1_the5ers'`** — falta ese feed en la librería de barras. Es el fallo más común. `python3 -m sqx.export.sync_bars` lo trae; son minutos por feed.
- **`FileNotFoundError` en `trades.parquet`** — la exportación no existe con esa fecha o ese nombre de databank. Ojo a los guiones bajos: `Retest Markets - Family` se exporta a `Retest_Markets_-_Family`.
- **Salida vacía durante media hora** — no está colgado: o le falta el `-u`, o lo canalizaste a `tail`. Mira `ps -o etime,time` del proceso; si el tiempo de CPU sube, está trabajando.

31. crossTF — ¿el edge aguanta si lo miras en un reloj más lento?

Qué pregunta responde

Coges una estrategia hecha en H1 y la miras en H4. Si lo que gana en H1 es una ventaja de verdad, debería seguir estando ahí cuando solo puedes actuar cada cuatro horas en vez de cada hora. Si desaparece, o era una ventaja de temporización tan fina que no la vas a poder ejecutar, o no era una ventaja.

El módulo hace esa comparación **dos veces, de dos maneras distintas**, porque son dos preguntas diferentes y solo una es un test de robustez:

	qué le haces a la estrategia	**qué significa que falle**
escalada	le divides los períodos entre 4 (una media de 20 horas sigue siendo de 20 horas)	**evidencia en contra de la estrategia**
sin escalar	la dejas igual (una media de 20 velas pasa a ser de 80 horas)	**nada sobre la estrategia** — eso pregunta si el mercado es autosemejante, que es otro tema

Cuándo lo usas, y cuándo no

Sí: sobre un puñado de supervivientes que ya han pasado la puerta OOS. Es un test caro por estrategia y su sitio es el final del embudo, no el principio.

No: sobre una población de miles recién construida. Y **no** como puerta binaria: la recomendación es leerlo como una puntuación más, porque H1 y H4 son la misma serie de precios remuestreada y la evidencia que aporta está muy correlacionada con la que ya te dio la puerta.

No para bajar a D1 desde H1. Se puede fabricar, pero el escalado divide entre 24 y los períodos reales se destruyen: medido sobre las tres madres de 2026-09-22, los parámetros se mueven entre un 140 % y un 586 % y **todos** quedan pegados al mínimo del constructor. Una variante así no es la estrategia escalada, es otra estrategia. D1 solo tiene sentido en la fila sin escalar de la madre.

Antes de empezar

1. Las madres, en una carpeta, como `.sqx`. No se tocan: el módulo solo lee.
2. `python3 -m core.assets XAUUSD` en verde, antes de tocar ninguna tarea de SQX.
3. Las tareas configuradas con `sqx.projects.crosstf`: una por timeframe, cada una con su databank, y todas sin condiciones de aceptación: queremos evidencia, no un filtro de selección.
4. El worker custodio parado antes de arrancar nada, y comprobar que no lo está usando otra sesión (`ls -lt user/projects` y el log).

Desde la ventana (lo normal)

Desde el 29-09 **no hay que hacer nada a mano entre el Cross Market y el Cross TF**. El orden es fijo: Cross Market → Cross TF → MC Retest. Al pulsar ▶ SQX del paso 11 (o «Correr workflow», o «Continuar workflow» sobre el databank de Cross Market), la ventana, con el worker parado:

1. coge las supervivientes que quedan en el databank de Cross Market;
2. las escala a los timeframes de la doctrina — desde H1: **M30, H4 y D1**; desde M30: H1 y H4 —;
3. vacía `CrossTF_Input` y copia ahí madres y hermanas;
4. escribe **una tarea por timeframe** (dueño, 2026-09-30): «CrossTF» corre el timeframe de la madre, y «CrossTF M30», «CrossTF H4» y «CrossTF D1» cada uno el suyo, con su propio databank (`CrossTF_M30`, `CrossTF_H4`, `CrossTF_D1`). **D1 corre con el motor de MetaTrader 4**;
5. arranca el worker, corre las cuatro tareas de un tirón, lo para y exporta cada databank.

Por qué una tarea por timeframe y no los bloques del crosscheck. SQX exporta las operaciones de todos los bloques de «Retest on additional markets» mezcladas por hora, y con el mismo símbolo no hay ninguna columna que diga de qué bloque es cada una. El código las separaba por el orden de los tickets, y eso asigna los bloques por **lo rápido que opera cada uno**, no por su timeframe: desde H1, las operaciones de M30 se leían como las de H1. Con una tarea por timeframe cada export es inequívoco (medido: M30 abre a la media hora en el 52 % de las operaciones, H4 solo a las 4, 8, 12, 16 y 20 h, D1 solo a las 00:00).

Por qué D1 con MetaTrader 4. En D1 la estrategia sólo entra a las 00:00, y la sesión de las fondeadas empieza a las 00:05: con el motor de MT5 no hacía ninguna operación. El motor de MT4 no pide sesión. Lo dice `crosstf.engines` en `assets/_build.yaml`.

Y antes del MC Retest (paso 13) copia a `CrossTF_Mothers` las madres que quedan en «CrossTF», sin sus hermanas escaladas: eso es lo que perturba el MC Retest.

Lo mismo a mano, con el worker parado:

```
python3 -m sqx.projects.crosstfload --project Test_USDJPY_donchianUpperCrossUp_H1 --role
custodian
python3 -m sqx.projects.crosstfload --project Test_USDJPY_donchianUpperCrossUp_H1 --role
custodian --mothers
```

Dos opciones nuevas (2026-10-02), para alimentar el paso desde fuera de la cadena:

flag	qué hace
`--from-folder CARPETA`	toma las madres de esa carpeta (todo `.sqx` que haya debajo) en vez del databank de Cross Market, y vacía antes los databanks de salida. Es lo que usa la calibración de criterios con `calib2/keep/<proyecto>/`
`--targets M30 H4`	escala y reprueba solo esos timeframes de la lista de la doctrina. La calibración de criterios corre **sin D1** («quítalo»): desde H1 todas las hermanas D1 salen pegadas al mínimo. La doctrina y la tarea «CrossTF D1» con MetaTrader 4 siguen como estaban

Salida real (corrida del 30-09, 30 madres):

```
30 madres x 3 timeframes -> 90 hermanas (48 con períodos pegados al mínimo: se leen solo sin escalar)
desde H1: M30, H4, D1 · una tarea por timeframe: H1, M30, H4, D1
120 .sqx en CrossTF_Input (madres + hermanas) · lote .../AlgoData/crosstf/<proyecto>/2026-09-30
«CrossTF M30»: M30 con MetaTrader5 (hedged) -> CrossTF_M30
«CrossTF H4»: H4 con MetaTrader5 (hedged) -> CrossTF_H4
«CrossTF D1»: D1 con MetaTrader4 -> CrossTF_D1
```

Un timeframe más rápido que el de la madre vale (H1 → M30): los períodos se doblan, sin redondeo y sin pegarse al mínimo. D1 desde H1 sale siempre pegado al mínimo, y la calibración de criterios del 2026-10-02 corre sin D1.

Cómo se ejecuta paso a paso

Son tres pasos: fabricar, correr, leer.

1 · Fabricar las hermanas escaladas

```
python3 -m sqx.variants.scale \
  --mothers ~/Desktop/AlgoData/projectsBackup/madres-2026-09-22 \
  --project XAUUSD --source H1 --targets H4
```

flag	obligatorio	qué hace
`--mothers`	sí	carpeta con los `.sqx` originales
`--project`	sí	de qué proyecto vienen; da nombre al árbol de salida
`--out`	no	solo para sacarlo fuera del sitio declarado; normalmente no se usa
`--source`	no (H1)	el timeframe en el que se construyeron las madres
`--targets`	no	a qué timeframes se escalan; por defecto los de la doctrina según `--source`: desde M30, H1 y H4; desde H1, **M30, H4 y D1**

Qué se escala (desde el 2026-10-02). Todo parámetro que sea una longitud en velas, y nada más: lo que el propio SQX marca como periodo en la estrategia (los tres del MACD, el slowing del estocástico, las longitudes de WaveTrend, QQE o BH Ergodic, que antes se quedaban sin tocar en el 65 % de las estrategias), la salida por número de velas, y en los bloques custom lo que diga la tabla `sqx/variants/lengths.yaml`, bloque a bloque. Niveles, multiplicadores, desviaciones, shifts y selectores no se tocan nunca. Un estocástico (5, 3, 3) de H1 pasa a (10, 6, 6) en M30. Un bloque custom que la tabla no conoce se deja como está y su hermana sale `unusable`.

Segundos. No toca SQX, se puede lanzar con la GUI abierta.

2 · Configurar las tareas, correrlas y exportarlas

```
python3 -m core.assets USDJPY                              # regla dura 5, bloqueante
python3 -m sqx.projects.crosstf USDJPY \
    --cfx ~/Desktop/SQX_w2/user/projects/Test_USDJPY_donchianUpperCrossUp_H1/project.cfx
```

flag	obligatorio	qué hace
`--cfx`	sí	el proyecto **custom**, nunca el `Retester` de serie (regla dura 10)
`--day`	no	el día del lote bajo el que guarda `blocks.json`; por defecto hoy

Pone «CrossTF» en el tramo `oos1` (desde el 2026-10-02 el paso lee **solo oos1**, a costes de oos1 y con la precisión «timeframe elegido»; antes era `build..oos1`), apaga su crosscheck y **todas** sus condiciones de aceptación (esto mide, no filtra: el corte se hace en Python y se aplica con `/curate`), y copia de ella una tarea por cada timeframe extra, cada una con su databank y su motor. Deja escrito en `blocks.json`, junto al lote, qué timeframe y qué databank es cada bloque: el estudio lee eso, nunca la doctrina de hoy. Imprime una línea por tarea:

```
CrossTF: H1, costes de USDJPY a `build..oos1`
«CrossTF M30»: M30 con MetaTrader5 (hedged) -> CrossTF_M30
«CrossTF H4»: H4 con MetaTrader5 (hedged) -> CrossTF_H4
«CrossTF D1»: D1 con MetaTrader4 -> CrossTF_D1
```

Luego cargas las madres *y* las hermanas en `CrossTF_Input` con el worker parado (`sqx.projects.crosstfload`, que ya llama a esto), corres las cuatro tareas y, al pararse el worker, cada databank se exporta solo.

3 · Leer

```
python3 -m studies.transfer.crossTF.report --project Test_USDJPY_donchianUpperCrossUp_H1 \
    --asset USDJPY --day 2026-09-30 --fabricated 2026-09-30
# la misma firma que los demás estudios: proyecto, databank (CrossTF por defecto), activo y
día.
# El export de CrossTF y los de CrossTF_<TF> (uno por timeframe, al lado) y el scaling.parquet
# salen de ahí; el feed, del activo en assets/.
```

Qué produce

Todo en `~/Desktop/AlgoData/crosstf/<proyecto>/<fecha>/`, fechado e inmutable (regla dura 7: los datos pesados nunca en el repo). **Su subcarpeta `sqx/` es la carga del databank**, tal cual (desde el 27-09; antes era la carpeta entera y SQX se quejaba en el log de `scaling.parquet`):

- `sqx/<madre>_ScaledH4.sqx` — una por madre y timeframe destino. 14 KB cada una.
- `sqx/<madre>.sqx` — **las madres, copiadas ahí al lado sin tocar**, porque el estudio necesita también sus celdas `baseline` y `control`. Así se carga una carpeta y no dos.
- `scaling.parquet`, fuera de `sqx/` — qué parámetro se movió y a cuánto, por hermana. **Es el registro de lo que se hizo**: sin él, un `.sqx` escalado es indistinguible de uno cualquiera.
- `--out` de `report.py`, opcional: el panel entero en Parquet.

Nada se sobrescribe salvo que apuntes dos veces a la misma carpeta.

Cómo se lee el resultado

El paso 1 imprime lo que ha movido:

```
3 mothers x 2 targets -> 6 siblings
                      name  ratio  n_scaled  max_rounding_shift  clamped
Strategy 10.13.25_ScaledH4    4.0         2            0.090909    False
Strategy 10.13.30_ScaledH4    4.0         6            0.200000    False
Strategy 29.30.36_ScaledH4    4.0         4            0.142857    False
Strategy 10.13.25_ScaledD1   24.0         2            1.400000     True
Strategy 10.13.30_ScaledD1   24.0         6            4.333333     True
Strategy 29.30.36_ScaledD1   24.0         4            5.857143     True

int parameters left untouched (10) -- review the whitelist:
  BBBarOpensShift1, CBlock_WssShfInt21, CloseDShift1, IchimokuSnkSpnCrsBshSgnStr1, ...
```

Dos columnas que se apuntan y se enseñan, y que **ya no descartan a ninguna hermana** (decisión tuya del 2026-10-02: «Redondea donde puedas, ya está»). Cada longitud se redondea al entero más cercano, con un mínimo de 2, y la hermana se lee como cualquier otra:

- **`max_rounding_shift`** — cuánto se desvió el parámetro peor por culpa del redondeo. Un período de 9 entre 4 es 2,25 y hay que escribir 2: eso es un 11 % de cambio **encima** del cambio de timeframe. **Menos es mejor**; queda como dato para medir después si las hermanas muy redondeadas se comportan distinto.
- **`clamped`** — un período que habría quedado por debajo de 2 y se ha pegado al mínimo. Queda apuntado; la fila se lee igual.

La última línea lista los parámetros enteros que el escalado **no** ha tocado. Están ahí para revisarlos, no porque haya fallado nada: la mayoría son shifts y selectores categóricos, que no deben escalarse.

El paso 3 imprime tres bloques. Primero la madre en cada timeframe sin escalar, **sin veredicto**, porque esa fila no juzga a la estrategia. Después las hermanas escaladas, cada una con su lectura:

lectura	qué pasó
`survives`	gana a su propio nulo de temporización en oos1 **y además** añade algo a lo que ya enseñó su madre (`z_new`)
`inherited`	gana dinero en oos1, pero no más de lo que el resultado de su madre ya predice
`fails`	pierde dinero en oos1
`no_data`	**sin datos**: menos de 30 operaciones en oos1, en la hermana o en su madre. No se juzga
`silent`	no hizo ninguna operación en oos1 en ese timeframe. No se juzga
`unusable`	lleva un bloque custom con una longitud sin verificar: no se pudo escalar. No se juzga. (El redondeo y el tope ya no descartan a nadie)

⚠ **Esta lectura es una propuesta de un agente del 2026-10-02, pendiente de tu aprobación.** Lo que sí decidiste ese día es que el paso lee solo oos1. Lo que la propuesta añade:

- **Solo oos1.** La estadística, el nulo y la p de cada celda son de las operaciones de oos1. La madre se eligió sobre el tramo de construcción: medida ahí salía siempre «significativa».
- `z`: a cuántas desviaciones del nulo queda la celda. Contra una estrategia sin ventaja y sin costes, 1,65 lo supera el 5 % y 2,33 el 1 %.
- `z_new`**: lo que la hermana añade a su madre.** Una hermana vuelve a leer el mismo activo en los mismos años. Si la madre pasó por suerte, su hermana «también pasa» entre el 14 % y el 40 % de las veces, no el 5 %. `z_new` descuenta lo que el resultado de la madre ya explica, usando lo parecidas que son las dos (la correlación de sus resultados mensuales): con él, el 5 % vuelve a ser el 5 %.
- **La suma por madre** (`combined_z_new`): lo que añaden juntas sus hermanas utilizables. Es el número sobre el que se escribirán las reglas del paso 12.
- **El control se enseña y no se juzga.** Era la lectura `control_failed`: se llevaba 29 de 46 hermanas reales y se invertía cuando la madre perdía dinero. La hermana con los periodos divididos entre cuatro, corrida en las velas de la madre, es otra estrategia, no un control.
- **Semilla.** `--seed N` repite los nulos exactamente; sin ella se sortea una nueva, que queda apuntada en el manifest (`nulls_seed`) y en la última línea que imprime el comando.

Un ejemplo completo

Fabricación real sobre las tres madres, mirando una por dentro antes y después:

```
MADRE H1  (7 miembros, 101.9 KB)          HIJA H4  (5 miembros, 14.4 KB)
   CBlock_KSBreakOPrd11        = 20          CBlock_KSBreakOPrd11        = 5
   CBlock_KSBreakOPrd21        = 50          CBlock_KSBreakOPrd21        = 12
   IchimokuSnkSpnCrsBshTnkPrd1 = 9           IchimokuSnkSpnCrsBshTnkPrd1 = 2
   IchimokuSnkSpnCrsBshKjnPrd1 = 26          IchimokuSnkSpnCrsBshKjnPrd1 = 6
   IchimokuSnkSpnCrsBshSnkPrd1 = 36          IchimokuSnkSpnCrsBshSnkPrd1 = 9
   IchimokuSnkSpnCrsBshSgnStr1 = 1           IchimokuSnkSpnCrsBshSgnStr1 = 1   <- intacto
   IchimokuSnkSpnCrsBshShf1    = 1           IchimokuSnkSpnCrsBshShf1    = 1   <- intacto
   ExitAfterBars1              = 10          ExitAfterBars1              = 2
   fingerprint = SI                          fingerprint = no, stamp = CTF001H4
```

Los períodos bajan; el shift y el selector de fuerza de señal se quedan. El `<Fingerprint>` heredado desaparece — si se quedara, el databank podría fundir todas las hermanas en una.

Qué NO te dice

- **No te dice que la estrategia sea buena.** Dice si su ventaja sobrevive a mirarla más despacio. Una estrategia mala puede pasar el test perfectamente.
- **La celda H4 no es «la misma estrategia, reloj más lento».** Por decisión del dueño (2026-09-23) solo se escalan períodos y salidas por barras. Los `StopLossCoef` y `ProfitTargetCoef` se escriben tal cual, y como multiplican un rango que crece al agregar velas, **el stop en H4 es físicamente el doble de ancho en precio**. Es una propiedad declarada de la comparación, no un fallo — pero no la describas como limpia.

- **H1 y H4 no son muestras independientes.** Una vela H4 está hecha de las mismas cuatro velas H1. Esto aporta mucha menos información que probar en otro mercado, y por eso el veredicto se apoya en el nulo de cada timeframe y no en «¿es rentable?».
- **No dice nada sobre datos no vistos.** Si el retest cae dentro de la ventana en la que se seleccionó la estrategia, sigue siendo ventana seleccionada.

Si algo falla

- **«CrossTF_D1 no tiene export».** Cada timeframe se exporta al parar el worker; si falló (el conductor estaba ocupado), pulsa ↻ en su databank o corre `python3 -m ui.daemon.loader.afterrun --role custodian`.
- **Una corrida anterior al 30-09 da números raros.** Entonces los timeframes eran bloques del crosscheck y se separaban por el orden de los tickets, que los confunde (M30 se leía como H1). El estudio lo avisa: esa corrida no es fiable; repítela.
- **Sale `unusable`.** Solo pasa cuando la estrategia lleva un bloque custom que la tabla `sqx/variants/lengths.yaml` no conoce o no ha podido verificar: hay que añadirlo ahí.